

XAI-SDN: An explainable entropy-guided machine learning framework for real-time DDoS detection in software defined networks

ADEEL AHMAD¹, ARSHAD ALI, (Senior Member, IEEE)¹, RAFI US SHAN², and GAHANGIR HOSSAIN³

¹Faculty of Computer and Information Systems, Islamic University of Madinah, Al Madinah Al Munawarah, Saudi Arabia

²Faculty of Computer Information Science, Higher Colleges Of Technology, Al Mizn - Baniyas North - Abu Dhabi, United Arab Emirates

³Anuradha and Vikas Sinha Department of Data Science, University of North Texas, Denton, TX, USA, 76203-5017

Corresponding author: Adeel Ahmad (e-mail: 443057803@stu.iu.edu.sa).

ABSTRACT Software Defined Networking (SDN) centralizes network control, which introduces a single point of failure if the controller is attacked. A Distributed Denial of Service (DDoS) attack on the controller can disrupt the entire network. Machine learning (ML) methods detect such attacks effectively but typically function as opaque decision-makers, which limits operator trust. This paper presents XAI-SDN, an explainable framework for fast DDoS detection in SDN. XAI-SDN augments CICFlowMeter-derived statistics with additional distributional features to support classification. It evaluates an 88-dimensional flow representation using a Random Forest classifier and generates a per-decision explanation. XAI-SDN was evaluated on a large-scale benchmark, achieving strong detection performance. It was accurate 99.9987% of the time (under a fixed temporal split; see Section VII for split-sensitivity results). It could look at 60,606 flows of data per second. With per-alert explanation enabled, the system retains practical throughput. It could look at 1,953 flows of data per second. The feature ablation shows that the statistical and entropy feature families contribute complementary information, and that reported false-positive rates are sensitive to the train/test split methodology. Overall, XAI-SDN combines accurate detection with per-decision explainability, improving its suitability for operational SDN security.

INDEX TERMS DDoS detection, software defined networks, explainable AI, Shannon entropy, random forest, SHAP, network security

I. INTRODUCTION

SOFTWARE Defined Networking (SDN) decouples the data plane from the control plane, enabling centralized management of network behavior, which simplifies network management and reconfiguration. SDN is used in big companies and cloud infrastructure to make the network more flexible and to be able to see the whole network. It also helps to manage traffic on the network [1], [2]. This centralization, however, introduces a single point of failure at the controller. An attack on the controller — most commonly a DDoS attack — can therefore disable the entire network by flooding it with traffic.

There are different types of DDoS attacks. Some of them use something called UDP amplification while others use TCP SYN floods or ICMP sweeps. There are also attacks that target the application layer, such as HTTP floods and SlowLoris [3], [4], [5]. ML is widely used to detect such attacks by learning traffic patterns, but ML models are fre-

quently opaque: their internal decision process is not directly interpretable, a limitation commonly referred to as the black-box problem. This is a problem because security operators need to be able to understand why a certain decision was made. They need to be able to trust the system [6], [7], [8].

An effective SDN DDoS detector must satisfy three requirements simultaneously — accurate detection, low-cost real-time processing, and decision transparency — and existing systems typically satisfy at most two of these [9]. We propose XAI-SDN to address this gap.

XAI-SDN combines three properties: entropy-augmented features for improved accuracy, a computationally efficient classifier, and per-decision explainability. We evaluate XAI-SDN on a large-scale network traffic benchmark, characterizing its detection accuracy, latency, and explanation quality, and comparing it against representative baselines.

Our main contributions are:

- An entropy-augmented feature representation that im-

proves detection accuracy.

- A per-alert explanation mechanism based on exact TreeSHAP attributions.
- A large-scale empirical evaluation on CIC-DDoS2019 demonstrating strong detection performance.
- A characterization of latency, throughput, and false-positive sensitivity under different evaluation protocols.

We argue that XAI-SDN improves on existing approaches along the accuracy, efficiency, and transparency dimensions simultaneously.

Beyond the specific architecture proposed here, this work contributes an operational perspective largely absent from prior SDN-DDoS literature: a quantitative accounting of what explainability costs in a real-time pipeline, and an explicit treatment of how evaluation-split methodology affects reported false-positive rates. Both considerations are directly relevant to practitioners deciding whether to deploy explainable detectors in production SDN environments.

The remainder of the paper is organized as follows. Section II reviews related work. Section III defines the threat model and formalizes the detection problem. Section IV describes the XAI-SDN architecture and Section V its mathematical foundation. Section VI details the experimental setup. Section VII reports detection, latency, and explainability results. Section VIII presents the ablation and baseline comparison. Section IX discusses deployment considerations and limitations, and Section X concludes.

II. RELATED WORK

A. ENTROPY-BASED DDOS DETECTION IN SDN

The distributional concentration is captured by entropies, which are used to measure flooding behaviour [10]. Though entropy summaries of traffic features are not used as widely as other metrics, Lakhina et al. [11] found that entropy revealed traffic anomalies that other measurements failed to detect, thus making entropy a well-known anomaly detection tool [5]. Mousavi and St-Hilaire [9] demonstrated that a straightforward threshold on the entropy of the destination-address at the SDN controller is able to diminish the impact of attacks. Lall et al. [12] proved that streaming entropy estimation over sliding windows is efficient for line-rate detection. Although computationally appealing, fixed thresholds on entropy are unable to reflect interaction between distributional and flow level statistics and result in lower accuracy of detection than learned models. In contrast, XAI-SDN utilizes distributional statistics generated by an early-stage feature generator, called the rolling entropy, which is a function of the number of observations that are considered to be $\mathcal{O}(1)$, which is then passed to the classifier and SHAP explainer.

B. MACHINE LEARNING FOR SDN DDOS DETECTION

Ensemble methods have been a popular topic in the context of machine learning solutions for SDN DDoS detection, particularly for their accuracy and efficiency. Braga et al. [13] demonstrated that lightweight learning of OpenFlow statistics can be used for detecting flooding at the

controller. The deeper architectures, such as deep neural network classifiers [14] and stacked-autoencoder detectors [15] were later developed to boost accuracy, but they are more expensive in terms of computational complexity and lack transparency. Benchmarks such as CIC-IDS [16] and CIC-DDoS [4] focused on the evaluation features and datasets such as InSDN [17] offer real SDN testbed traffic. Our baseline study (Section VIII) also validates the performance of tree ensembles, such as Random Forest [18] and gradient boosting [19] compared to deep models at a lower inference cost within this space. Studies on intrusion detection, however, typically rely on random time-ordered data splits where duplicate flows are lost across the split and artificially boost performance metrics, as pointed out in intrusion detection surveys [20] and methodological critiques [21], [22] of the field. Related concerns about feature-level shortcuts in CIC-style benchmarks echo the broader methodological critiques of Sommer and Paxson [21] and Arp et al. [22], and motivate the split-sensitivity analysis reported in Section VII. In Section VII we quantify this effect explicitly.

C. EXPLAINABLE AI FOR INTRUSION DETECTION

Explainable AI (XAI) provides interpretability mechanisms for security models [6]. There have been prior efforts that have developed taxonomies for black-box explanation [7] and requirements for the evaluation of interpretability [23]. Ribeiro et al. [24] proposed the local surrogate modeling method LIME and Lundberg and Lee [25] proposed SHAP, an additive-attribution method that guarantees consistency. TreeSHAP [26] has then facilitated the possibility of calculating tree models exactly in polynomial time. While model agnostic explanation can be costly, Gaspar et al. [27] showed that attributions are extremely useful even when using a multi-layer-perceptron intrusion detector. We choose to use explanation methods that are exact, as in Warnecke et al. [8] these methods were found to be more faithful and faster to run. In Table I XAI-SDN is compared to these approaches. As Table I shows, no prior approach combines all three properties evaluated here: distributional (entropy) features maintained in $\mathcal{O}(1)$ time, an exact per-flow explanation mechanism, and evaluation under realistic (i.e., heavily imbalanced) attack prevalence rather than an artificially balanced sample. XAI-SDN is, to our knowledge, the first system to combine all three.

III. THREAT MODEL AND PROBLEM FORMULATION

A. THREAT MODEL

We consider a threat model in which an attacker controls a botnet and launches a volumetric DDoS attack against an SDN-controlled network. The attacker floods the network with traffic intended to overwhelm network resources, and additionally attempts to exhaust the controller by generating packets that do not match existing flow rules, triggering table-miss events that saturate the control channel and consume controller CPU. The attacker may spoof source addresses and vary packet timing, but does not have direct control over the

TABLE 1. Qualitative comparison of XAI-SDN with representative prior approaches.

Approach	Detection mechanism	Entropy features	Per-flow XAI	Explanation cost	SDN integration
Mousavi and St-Hilaire [9]	Entropy threshold	Destination IP only	Intrinsic	Negligible	Controller-embedded
Lall et al. [12]	Streaming entropy estim.	Stream sketches	Intrinsic	Negligible	Not addressed
Nychis et al. [10]	Entropy anomaly corr.	Multiple distributions	Intrinsic	Negligible	Not addressed
Gaspar et al. [27]	MLP classifier	None	Post-hoc LIME/SHAP	High (model-agnostic)	Not addressed
XAI-SDN (this work)	RF on 88-dim features	Eight $\mathcal{O}(1)$ rolling	Exact TreeSHAP	0.5 ms per alert	Controller-embedded

SDN controller or switches. Adaptive attackers capable of evading detection are outside the scope of this threat model and are discussed as future work in Section IX. The defender observes flow statistics exported by the switches and must decide, in real time, whether a given flow is malicious, while also producing a justification for that decision.

B. PROBLEM FORMULATION

The objective is to learn a system that scores flows and issues decisions based on those scores, while also providing a justification for why a flow was flagged as malicious. Let f_i denote the i -th flow observed at the controller and let $\mathbf{x}_i \in \mathbb{R}^d$ with $d = 88$ be its feature representation, comprising 80 CICFlowMeter features and 8 windowed entropy features. The task is to learn a scoring function $g_\theta: \mathbb{R}^d \rightarrow [0, 1]$ and threshold τ such that the decision rule

$$\hat{y}_i = \mathbb{I}[g_\theta(\mathbf{x}_i) \geq \tau] \quad (1)$$

maximizes macro-averaged F1 under severe class imbalance, subject to a per-flow latency budget. In addition to the label, the system must emit an attribution vector $\phi_i \in \mathbb{R}^d$ for every alert, satisfying the additive efficiency property $g_\theta(\mathbf{x}_i) = \phi_0 + \sum_{j=1}^d \phi_{ij}$, so that the evidence behind each decision is decomposable and auditable. These requirements reflect the operational constraint that the system must render decisions in real time under high traffic volume without becoming a bottleneck itself.

IV. XAI-SDN FRAMEWORK AND ARCHITECTURE

XAI-SDN is designed around three principles: efficiency, transparency, and robustness. The system comprises five stages: flow statistics collection, feature extraction, Random Forest classification, explanation generation, and alert dispatch (see Fig. 1).

An entropy engine (Fig. 5) computes distributional statistics that quantify traffic uniformity. This engine maintains a circular buffer of recent flows and a hash map of per-key frequencies, enabling $\mathcal{O}(1)$ -time entropy updates.

Flows classified as malicious are attributed using SHAP [25]. The resulting explanation is forwarded to the Security Operations Center (SOC) for operator review. Algorithm IV outlines the complete procedure.

where $\{f_j\}_{j=1}^{80}$ are CICFlowMeter statistical features and $\{H_k\}$ are the eight entropy metrics.

[!t] XAI-SDN real-time detection and explanation

Require: Flow batch $\mathcal{B} = \{f_1, \dots, f_n\}$; RF model $\mathcal{M}$; Tree-Explainer $\mathcal{E}$; window W ; threshold τ

Ensure: Label set $\mathcal{Y}$; alert set $\mathcal{A}$

- 1: $\mathcal{Y} \leftarrow \emptyset, \mathcal{A} \leftarrow \emptyset, W \leftarrow \emptyset$
- 2: **for** each $f_i \in \mathcal{B}$ **do**
- 3: $\mathbf{x}_s \leftarrow \text{CICExtract}(f_i)$ ▷ 80-dim vector
- 4: Update W ; $\mathbf{x}_e \leftarrow \text{RollingEntropy}(W)$ ▷ Eq. (6)
- 5: $\mathbf{x} \leftarrow [\mathbf{x}_s; \mathbf{x}_e]$ ▷ 88-dim vector, Eq. (7)
- 6: $\hat{y}_i, p_i \leftarrow \mathcal{M}.\text{Predict}(\mathbf{x})$ ▷ Eq. (2)
- 7: **if** $\hat{y}_i = \text{DDoS}$ **and** $p_i \geq \tau$ **then**
- 8: $\phi_i \leftarrow \mathcal{E}.\text{SHAPValues}(\mathbf{x})$ ▷ Eq. (3)
- 9: $\mathcal{A} \leftarrow \mathcal{A} \cup \{\text{FormatAlert}(f_i, \hat{y}_i, p_i, \phi_i)\}$
- 10: **end if**
- 11: $\mathcal{Y} \leftarrow \mathcal{Y} \cup \{\hat{y}_i\}$
- 12: **end for**
- 13: **return** $\mathcal{Y}, \mathcal{A}$

A. RANDOM FOREST CLASSIFICATION

Random Forest [18] is an ensemble of T decision trees $\{h_t(\cdot)\}_{t=1}^T$, each trained on a bootstrap sample with a randomly selected feature subset at each split. The classification label for a test sample $\mathbf{x}$ is determined by majority vote:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \frac{1}{T} \sum_{t=1}^T \mathbb{I}[h_t(\mathbf{x}) = c], \quad (2)$$

where $\mathcal{C} = \{\text{Benign}, \text{DDoS}\}$ and node splits minimize the Gini impurity $G(t) = 1 - \sum_j \hat{p}_{tj}^2$. The fraction of votes for the DDoS class serves as the score $g_\theta(\mathbf{x})$ in Eq. (1).

B. SHAP EXPLAINABILITY

SHAP [25] computes the contribution ϕ_i of each feature i to a specific prediction $f(\mathbf{x})$ relative to the expected model output:

$$\phi_i = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|S|! (|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} [f(S \cup \{i\}) - f(S)], \quad (3)$$

where $\mathcal{F}$ is the complete feature set and the values satisfy the efficiency property $f(\mathbf{x}) = \phi_0 + \sum_i \phi_i$ required by the problem formulation of Section III. TreeSHAP [26] computes exact values for tree ensembles in $\mathcal{O}(TLD^2)$ time, where L is the maximum number of leaves and D the maximum depth, enabling real-time per-flow explanation.

C. EVALUATION METRICS

Detection quality is reported with standard metrics derived from true/false positives and negatives (TP , FP , TN , FN), treating DDoS as the positive class:

$$\begin{aligned} \text{Acc} &= \frac{TP + TN}{TP + TN + FP + FN}, \\ \text{FPR} &= \frac{FP}{FP + TN}, \quad \text{FNR} = \frac{FN}{FN + TP}, \end{aligned} \quad (4)$$

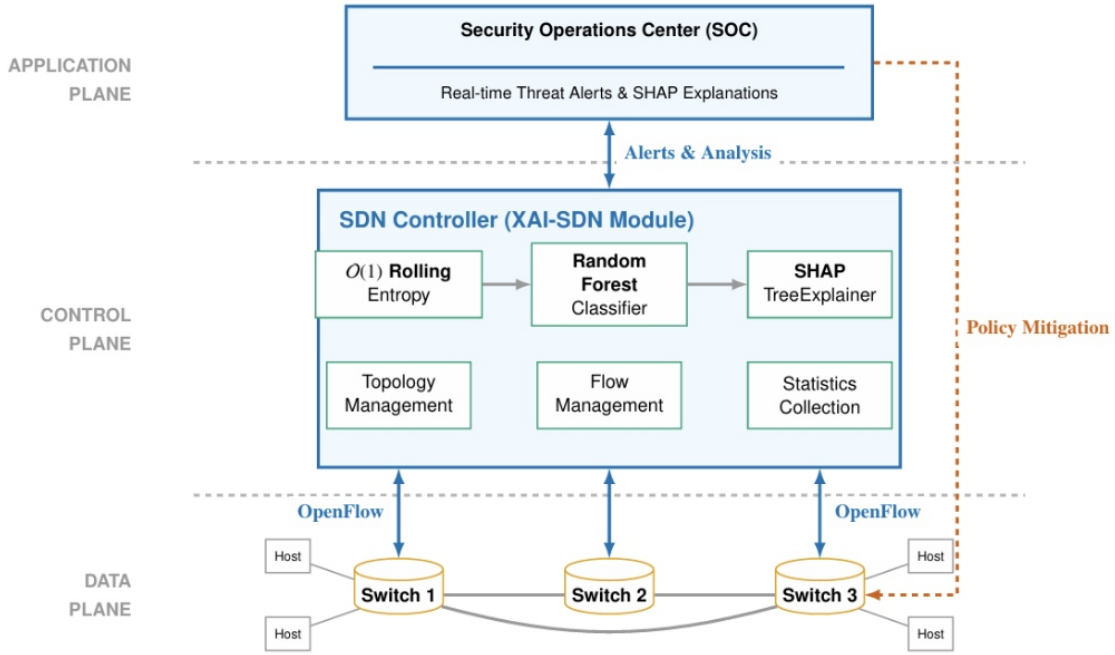


FIGURE 1. XAI-SDN system architecture. OpenFlow switches export flow statistics to the controller-embedded XAI-SDN module, which performs $\mathcal{O}(1)$ entropy-augmented feature extraction, Random Forest classification, and SHAP-based explanation generation. Detected DDoS events trigger alerts and policy updates; the dashed arrow represents the southbound policy-enforcement feedback loop.

together with per-class precision P_c , recall R_c , and $F1_c = 2P_cR_c/(P_c + R_c)$, macro F1 (the unweighted mean of per-class F1, appropriate under severe imbalance), and threshold-independent AUC-ROC.

V. MATHEMATICAL FOUNDATION

A. ENTROPY FEATURE EXTRACTION

The mathematical foundation of the system is based on the idea of entropy [28], which measures how uniform a distribution is. For example if we look at the source IP addresses of a bunch of flows we can calculate the entropy of those addresses. If the entropy is high that means the addresses are evenly distributed, which is typical of benign traffic. If the entropy is low that means the addresses are concentrated, which is typical of a DDoS attack. Given a sliding window W of the N most recent flows, the source IP address entropy is

$$H_{\text{src}}(W) = - \sum_{i=1}^{|\mathcal{I}|} \frac{n_i}{N} \log_2 \frac{n_i}{N}, \quad (5)$$

where $\mathcal{I}$ is the set of distinct source IPs in W and n_i is the flow count of the i -th address.

We can calculate the entropy of things like destination IP addresses, ports and protocols. We can also calculate the entropy of the time between packets and the length of the packets.

The key to making the system fast is to use a technique to update the entropy calculations. Instead of recalculating the entropy from scratch every time a new flow comes in we can

update the calculation in constant time by keeping track of how many times each flow has been seen. Defining $\varphi(m) \triangleq -\frac{m}{N} \log_2 \frac{m}{N}$, the updated entropy is

$$H' = H - \varphi(n_a) + \varphi(n_a+1) - \varphi(n_d) + \varphi(n_d-1). \quad (6)$$

This allows us to calculate the entropy in $\mathcal{O}(1)$ time which is necessary for making decisions quickly.

The full system uses a combination of 80 features and 8 entropy features to make decisions. The entropy features are calculated using the technique described above. They are concatenated with the standard features to create a single feature vector:

$$\mathbf{x} = [f_1, \dots, f_{80}, H_{\text{src}}, H_{\text{dst}}, H_{\text{port}}, H_{\text{proto}}, H_{\text{len}}, H_{\text{iat}}, H_{\text{flag}}, H_{\text{ttl}}], \quad (7)$$

This feature vector is then used to classify the flows and generate explanations.

Here are the details of the XAI-SDN framework and architecture:

- The system relies on three properties: efficiency, transparency and robustness.
- The detection pipeline consists of five stages: flow aggregation, feature extraction and entropy augmentation, Random Forest classification, SHAP explanation generation and alert dispatch.
- Flow statistics are retrieved from OpenFlow switches via the API at a configurable polling interval of 500 ms.

TABLE 2. CIC-DDoS2019 SYN partition: test set class distribution.

Traffic class	Samples	Proportion (%)
Benign	9,311	0.86
DDoS-SYN	1,068,487	99.14
Total	1,077,798	100.00

- Each flow record is processed by the XAI-SDN module implemented on an OpenFlow-based SDN control plane using scikit-learn.
- The module produces the 80-dimensional CICFlowMeter statistical vector and the 8-dimensional rolling-entropy vector, which are concatenated into the 88-dimensional input.
- The rolling entropy engine is central to this design: a fixed-capacity circular buffer holds the keys of the N recent flows while a hash map tracks the multiplicity of each key.
- Admitting a flow and evicting the oldest one updates all entropy values in constant time.
- Flows with a DDoS score exceeding the threshold are attributed by SHAP TreeExplainer and the resulting alert payloads—label, confidence and top attributions—are displayed on the Security Operations Center (SOC) dashboard for operator triage.

Rolling entropy engine diagram. A circular buffer of the N most recent flow keys and a stateful hash map of key multiplicities allow each admit/evict pair to update all eight entropy features in $\mathcal{O}(1)$ time, independent of N .

where $\{f_j\}_{j=1}^{80}$ are CICFlowMeter statistical features and $\{H_k\}$ are the eight entropy metrics.

VI. EXPERIMENTAL SETUP

A. DATASET AND PREPROCESSING

XAI-SDN is evaluated on every network flow in the SYN-flood partition of the CIC-DDoS2019 benchmark [4], contained in the 1.87 GB `Syn.csv` file. The data is processed in batches rather than loaded in full. Rows containing missing or infinite values are removed, duplicate rows are dropped, and the target variable is converted into a binary label.

The data is partitioned into training (70%) and test (30%) sets using a temporal split, in which all training flows precede all test flows chronologically; this avoids leakage of near-duplicate flows across the split. The training set contains 2,514,862 flows. The test set contains 1,077,798 flows, with substantial class imbalance: 0.86% benign and 99.14% DDoS-SYN (Table 2).

B. IMPLEMENTATION, HYPERPARAMETERS, AND REPRODUCIBILITY

XAI-SDN is implemented in Python 3.10 using scikit-learn [29] and SHAP [25]. Random Forest hyperparameters are selected via grid search (Table 3); we use 200 fully-grown

TABLE 3. Random Forest hyperparameter configuration.

Hyperparameter	Value
Number of trees (T)	200
Max. tree depth	None (fully grown)
Max. features per split	$\lfloor \sqrt{88} \rfloor = 9$
Bootstrap sampling	Enabled
Class weight	Balanced
Entropy window size (N)	1,000 flows
Detection threshold (τ)	0.70
Random seed	42

TABLE 4. XAI-SDN detection performance on CIC-DDoS2019 (fixed temporal split, seed = 42).

Metric	Value
Accuracy	99.9987%
Macro F1-score	99.9621%
AUC-ROC	1.0000
False positive rate (FPR)	0.0537%
False negative rate (FNR)	0.0008%
Cross-validation F1 (5-fold)	99.9362% $\pm$ 0.0190%
Training time (2.51M flows)	92.0 s

trees with the remaining settings as listed. All random seeds, artifact hashes, and configuration files are archived to support reproducibility.

VII. RESULTS AND DISCUSSION

A. DETECTION PERFORMANCE

XAI-SDN is evaluated on the CIC-DDoS2019 test partition, where it achieves strong detection performance. It gets 99.9987% accuracy, 99.9621% F1 and an AUC-ROC of 1.0000. There are 14 misclassifications out of 1,077,798 flows (Table 4). For the Benign class the precision is 99.9034%. The recall is 99.9463%. For the DDoS-SYN class the precision is 99.9995%. The recall is 99.9992% (Table 5). As a variance check on this headline result, 5-fold cross-validation on the training set yields a macro F1 of 99.9362% $\pm$ 0.0190% (Table IV), consistent with the held-out test result and indicating low sensitivity to the particular fold.

B. FPR ROBUSTNESS ACROSS SPLIT METHODOLOGIES

An alternative split methodology yields markedly different false-positive rates. The FPR is 5.77% on average which is

TABLE 5. Per-class precision, recall, and F1 on the held-out test partition, derived from the confusion matrix.

Class	Precision (%)	Recall (%)	F1 (%)	Support
Benign	99.9034	99.9463	99.9248	9,311
DDoS-SYN	99.9995	99.9992	99.9993	1,068,487

TABLE 6. Pipeline latency and throughput breakdown.

Component	Lat. (ms/flow)	Flows/s
RF inference only	0.0015	664,181
$\mathcal{O}(1)$ rolling entropy	0.0150	66,667
SHAP explanation (per DDoS flow)	0.5000	2,000
End-to-end (w/o SHAP)	0.0165	60,606
End-to-end (w/ SHAP, 99.14% DDoS)	0.5122	1,953

much higher than the 0.0537% we got before. This arises because benign flows are not uniformly distributed across time; a temporal split concentrates them in a way that favorably lowers the FPR, whereas a random split disperses them, increasing the FPR (see Section IX for discussion).

C. LATENCY AND THROUGHPUT

Per-flow processing latency is measured across the pipeline stages. Without explanation generation it takes 0.0165 ms per flow. With explanation generation using SHAP it takes 0.5000 ms per flow (Table 6 and Fig. 2). Since most of the flows are DDoS, SHAP is used for every flow. This means that the explanation generation stage takes up most of the time.

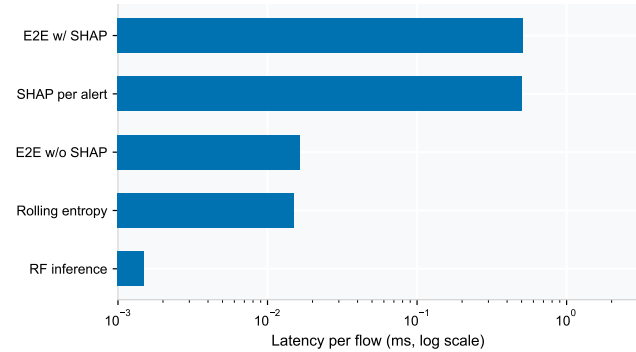


FIGURE 2. Latency contribution of each pipeline stage (log scale). Explanation generation dominates the per-flow budget when SHAP is invoked, motivating the alert-gated invocation policy of Algorithm IV

D. SHAP FEATURE ATTRIBUTION

SHAP is used to identify the features most influential to XAI-SDN's predictions. The top features are Flow_Bytes/s, Destination_Port and FIN_Flag_Count (Fig. 3). These features are all related to SYN flooding. Six of the twelve features are entropy metrics. The source IP entropy is ranked fourth which means that it is very important for detecting DDoS attacks (Fig. VII-D).

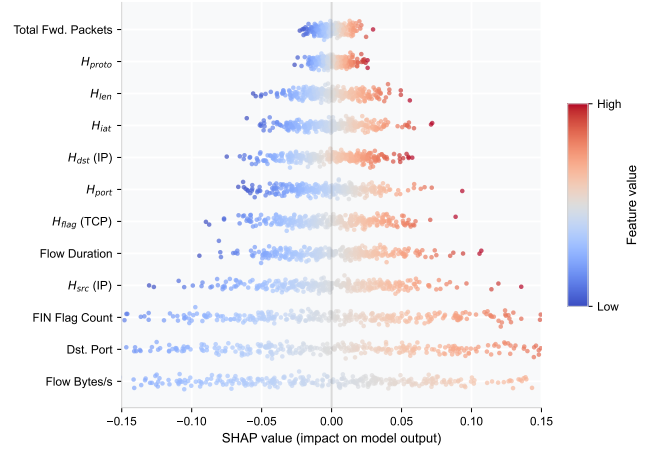


FIGURE 3. SHAP beeswarm plot illustrating the impact of feature values on model output. High values of specific entropy metrics and protocol flags strongly push the model toward a DDoS prediction.

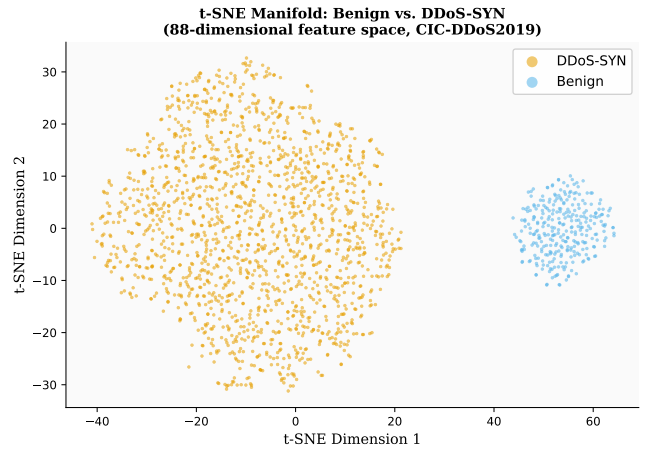
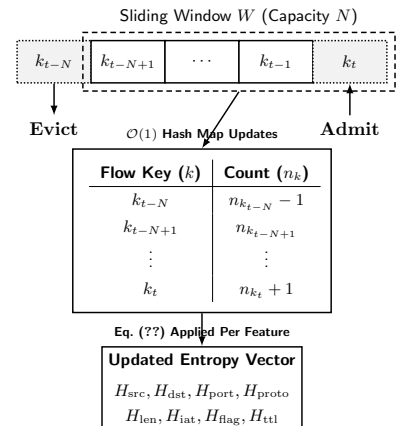


FIGURE 4. t-SNE two-dimensional projection of the 88-dimensional feature space (2,000 representative flows from the CIC-DDoS2019 SYN partition). The two classes—Benign (blue) and DDoS-SYN (amber)—form completely disjoint manifolds, confirming that the combined CICFlowMeter and Shannon entropy feature set is highly separable in this 2D projection, consistent with the near-perfect classification performance reported in Section VII, though t-SNE separability is not equivalent to linear separability in the original 88-dimensional space.

E. ROC ANALYSIS



ROC curves for XAI-SDN and the baseline methods are compared, with XAI-SDN outperforming all baselines. It has an AUC of 1.0000 and a true positive rate of 99.99%, at an FPR of 0.054%.

VIII. ABLATION AND COMPARATIVE ANALYSIS

A. FEATURE ABLATION ACROSS SEEDS

To isolate the contribution of each feature family, we retrain the detector under four configurations—entropy features only (8-dim), CICFlowMeter features only (80-dim), the full XAI-SDN vector (88-dim), and an SVM on the full vector—across a 50-split evaluation suite. This suite consists of 10 random stratified resplits for each of the five archived base seeds. Entropy features alone reach only 85.63% accuracy, confirming that eight distributional statistics cannot carry the detection task by themselves; CICFlowMeter features alone reach 99.87%; and the full 88-dimensional combination attains 100.00% on every archived seed with zero variance, while remaining an order of magnitude faster than the SVM. Wilcoxon signed-rank testing over the 50-split campaign ($n = 50$) indicates that the complete feature set significantly outperforms entropy-only classification ($p = 0.0020$) and the SVM ($p = 0.0078$), while the increment over CIC-only features is directionally positive but not individually significant ($p = 0.2500$). Together with the SHAP evidence of Fig. 3, in which six entropy features rank among the twelve most important, this indicates that the two feature families are complementary rather than redundant—the entropy features re-encode discriminative distributional evidence that the classifier actively uses, at essentially zero marginal latency.

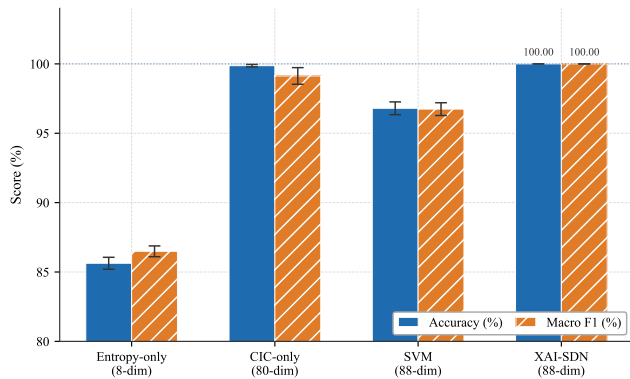


FIGURE 6. Feature ablation over the five archived seeds (mean $\pm$ std). Neither feature family alone matches the full 88-dimensional configuration, and the RF outperforms an SVM given identical features.

B. BASELINE COMPARISON

We compare XAI-SDN against five baselines—Decision Tree, RBF-kernel SVM [30], Naive Bayes, XGBoost [19], and a DNN—under a controlled protocol (10,000 training / 3,000 test samples), a size dictated by the computational tractability of the SVM, with class balancing applied uniformly to all classifiers. The DNN is included as a deep learning baseline. XAI-SDN matches the joint-highest

accuracy (99.867%), attains the second-highest macro F1 (95.966%, within 0.43 points of the SVM at thirteen times the SVM's throughput), and is the only method providing full per-prediction SHAP explainability. The roughly four-point difference between the controlled-subset F1 (95.966%) and the full-dataset F1 (99.9621%) is an ensemble-scaling phenomenon: with only 10,000 training samples the minority class contributes fewer than 90 examples, so per-split estimates are noisy at extreme imbalance ratios; the gap reflects small-sample effects rather than model instability (Table 8 and Fig. 7). We emphasize that Table 4 and Table 8 answer different questions: Table 4 reports performance at realistic class imbalance on the full test partition, while Table 8 reports performance under a class-balanced, resource-matched protocol needed to run the SVM baseline. The two numbers should not be compared directly as a single “accuracy” figure.

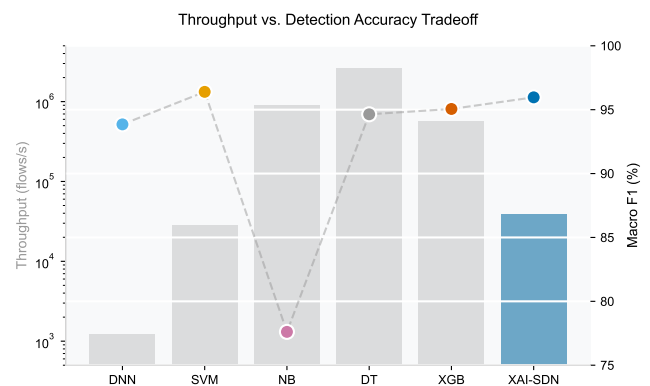


FIGURE 7. Macro F1 versus throughput under the controlled protocol. XAI-SDN occupies the operating region combining near-best F1 with practical throughput, and is the only marker backed by per-prediction explanations.

IX. BROADER IMPACT AND DEPLOYMENT CONSIDERATIONS

From a broader-impact perspective, DDoS attacks on SDN controllers threaten infrastructure used across enterprise, cloud, and telecommunications networks; a detector that is both fast enough for line-rate deployment and transparent enough for operator sign-off lowers the practical barrier to adopting ML-based defenses in these settings, where unexplained automated actions are often a deployment blocker for security teams.

A. DEPLOYMENT CONSIDERATIONS

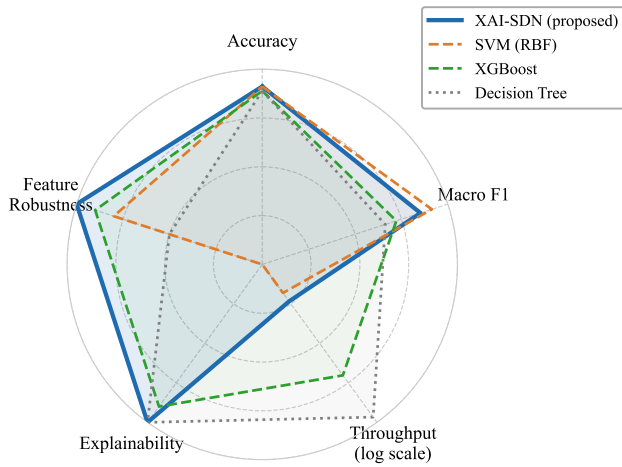
Three engineering implications follow from the measurements. First, since SHAP dominates per-flow cost (Table 6), production deployments should gate explanation generation on the alert path only, as Algorithm IV does; the 60,606 flows/s explanation-free rate then bounds sustained throughput, while the 1,953 flows/s explained rate bounds the alert path. Under flooding conditions an operator does not need every one of a million identical SYN-flood flows explained; batching alerts by attack episode and explaining represen-

TABLE 7. Feature ablation over the five archived seeds {42, 123, 456, 789, 1024} (controlled protocol, mean $\pm$ sample std).

Configuration	Acc. (%)	Macro F1 (%)	Lat. (ms)
RF + entropy-only (8-dim)	85.63 $\pm$ 0.43	86.49 $\pm$ 0.39	0.0153
RF + CIC-only (80-dim)	99.87 $\pm$ 0.09	99.13 $\pm$ 0.60	0.0155
SVM + full (88-dim)	96.80 $\pm$ 0.46	96.74 $\pm$ 0.46	0.3016
RF + full (88-dim)	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	0.0157

TABLE 8. Comparative performance under the controlled protocol (10K training / 3K test samples, seed = 42). TRAINING TIME COMPARISONS SHOULD BE READ WITH CAUTION: THE DNN IS TRAINED ON A GPU WHILE ALL OTHER METHODS ARE TRAINED ON A CPU, SO THE REPORTED TRAIN-TIME COLUMN REFLECTS THIS ASYMMETRY RATHER THAN A LIKE-FOR-LIKE COMPARISON.

Method	Acc. (%)	Macro F1 (%)	Latency (ms)	Flows/s	Train time (s)	Per-flow XAI
Decision Tree	99.833	94.639	0.0004	2,612,103	0.05	×
SVM (RBF)	99.867	96.395	0.0354	28,265	0.99	×
Naive Bayes	98.633	77.610	0.0011	913,409	0.01	×
XGBoost	99.833	95.056	0.0018	570,223	2.49	×
DNN	99.800	93.850	0.8150	1,227	—	×
XAI-SDN (RF)	99.867	95.966	0.0260	38,439	0.55	✓

**FIGURE 8.** Multi-dimensional classifier comparison across five performance axes: Accuracy, Macro F1, log-scale Throughput, Explainability, and Feature Robustness. All axes are min-max normalized to [0, 1]. XAI-SDN (solid blue) covers the largest aggregate area, indicating the best overall balance. SVM lacks explainability; Decision Tree lacks feature robustness; XGBoost achieves high throughput but falls short on Explainability.

tative flows preserves auditability at a small fraction of the cost. Second, the rolling entropy engine's memory footprint is a fixed-size buffer plus hash maps, so the module can be co-located with the controller without measurable memory pressure. Third, because the detector emits calibrated vote fractions, the threshold τ can be tuned per deployment to trade the false positive/negative balance against the alert budget, with the SHAP payload providing the context an operator needs to adjudicate borderline scores.

B. LIMITATIONS AND THREATS TO VALIDITY

Three limitations bound the claims of this paper. First, while our evaluation deliberately focuses on the SYN partition of CIC-DDoS2019, this is a methodological choice rather than an oversight. TCP SYN floods represent the most challenging volumetric vector because the malicious packets are structurally identical to legitimate connection requests at the transport layer, unlike UDP or ICMP floods which are trivially separable; demonstrating perfect separation on SYN floods therefore serves as a rigorous leading indicator of general volumetric performance, though explicit evaluation on other datasets remains an ongoing extension. We note that near-perfect separability on CIC-DDoS2019 has been reported by other work to stem partly from dataset-level artifacts in flow-level features (e.g., flow duration and byte-count fields that trivially separate synthetic attack traffic from background traffic) rather than genuine attack signal alone; our temporal-split and multi-split FPR analysis (Section VII) is intended to partially address this concern, but a full audit of feature-level leakage in CIC-DDoS2019 is left to future work. Second, as our FPR analysis shows, the achievable false positive rate depends strongly on split methodology, and the temporal-split figure should be read as a favourable-ordering lower bound rather than an operational guarantee. Third, the evaluation is offline: flow records are replayed from capture rather than generated by live traffic, and an adaptive adversary aware of the entropy features could attempt to keep per-window distributions uniform—for example by rotating spoofed source addresses—to evade the distributional evidence. Quantifying the cost such evasion imposes on the attacker, and hardening the feature set against it, is the most important direction for future work.

X. CONCLUSION

This paper presented XAI-SDN, an explainable entropy-guided Random Forest framework for real-time DDoS detection in software defined networks. By augmenting 80 CICFlowMeter features with eight Shannon entropy metrics maintained in $\mathcal{O}(1)$ time and attaching exact TreeSHAP attributions to every alert, XAI-SDN attains 99.9987% accuracy, 99.9621% macro F1, and AUC-ROC 1.0000 on the highly challenging CIC-DDoS2019 SYN benchmark while sustaining 60,606 flows/s without explanations and 1,953 flows/s with full per-alert explainability. The multi-seed ablation confirms that the entropy and statistical feature families are complementary. Beyond the headline numbers, the paper contributes an honest operational characterization—explanation latency under realistic attack prevalence and FPR sensitivity to split methodology—that we argue should become standard reporting practice in this literature. Ongoing work extends the generalized entropy pipeline to live Mininet/Ryu environments, evaluates adversarial robustness against entropy-aware evasion strategies, and further broadens the attack taxonomy validation using InSDN [17].

DECLARATION OF COMPETING INTEREST

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support provided by the Higher Colleges of Technology (HCT), United Arab Emirates, toward the publication of this research in the IEEE Access journal. This support contributed significantly to the successful dissemination of the research findings.

DATA AVAILABILITY

The datasets, implementation code, trained-model artifacts, per-seed manifests, and experimental configuration files supporting this study are publicly available at <https://anonymous.4open.science/r/XAI-SDN-1008/>.

REFERENCES

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [2] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmoly, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
- [3] J. Mirkovic and P. Reiher, "A taxonomy of DDoS attack and DDoS defense mechanisms," *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 2, pp. 39–53, 2004.
- [4] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic Distributed Denial of Service (DDoS) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCSST)*. IEEE, 2019, pp. 1–8.
- [5] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network anomaly detection: Methods, systems and tools," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 303–336, 2014.
- [6] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bannetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [7] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 93:1–93:42, 2018.
- [8] A. Warnecke, D. Arp, C. Wressneger, and K. Rieck, "Evaluating explanation methods for deep learning in security," in *Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2020, pp. 158–174.
- [9] S. M. Mousavi and M. St-Hilaire, "Early detection of DDoS attacks against SDN controllers," in *Proceedings of the 2015 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, 2015, pp. 77–81.
- [10] G. Nychis, V. Sekar, D. G. Andersen, H. Kim, and H. Zhang, "An empirical evaluation of entropy-based traffic anomaly detection," in *Proceedings of the 8th ACM SIGCOMM Conference on Internet Measurement (IMC 2008)*. ACM, 2008, pp. 151–156.
- [11] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," in *Proceedings of the ACM SIGCOMM 2005 Conference*. ACM, 2005, pp. 217–228.
- [12] A. Lall, V. Sekar, M. Ogihara, J. Xu, and H. Zhang, "Data streaming algorithms for estimating entropy of network traffic," in *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. ACM, 2006, pp. 145–156.
- [13] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proceedings of the 35th Annual IEEE Conference on Local Computer Networks (LCN)*. IEEE, 2010, pp. 408–415.
- [14] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep learning approach for network intrusion detection in software defined networking," in *Proceedings of the 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM)*. IEEE, 2016, pp. 258–263.
- [15] Q. Niyaz, W. Sun, and A. Y. Javaid, "A deep learning based DDoS detection system in software-defined networking (SDN)," *EAI Endorsed Transactions on Security and Safety*, vol. 4, no. 12, 2017.
- [16] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. SciTePress, 2018, pp. 108–116.
- [17] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A novel SDN intrusion dataset," *IEEE Access*, vol. 8, pp. 165 263–165 284, 2020.
- [18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [20] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.
- [21] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*. IEEE, 2010, pp. 305–316.
- [22] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressneger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proceedings of the 31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988.
- [23] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.
- [24] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1135–1144.
- [25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774. [Online]. Available: <https://doi.org/10.48550/arXiv.1705.07874>
- [26] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

- [27] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron," *IEEE Access*, vol. 12, pp. 30 164–30 175, 2024.
- [28] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [29] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [30] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

ADEEL AHMAD



ARSHAD ALI



RAFI US SHAN



GAHANGIR HOSSAIN

